Mark-and-Sweep Garbage Collection in Block-MiniJava

Logic, Formalization, and Implementation Mapping

Research note — Block-MiniJava

2026-07-20

Abstract

Model A of Block-MiniJava's abstract machine (§1) is a faithful, mutable-heap encoding of MiniJava: every `new` grows a store that nothing ever shrinks on its own. `src/core/semantics/gc.ts` and `src/core/semantics/reachability.ts` add a classical *tracing, mark-and-sweep* collector on top of that heap, composed as its own machine transition rather than folded into `step`. This note gives the collector's logic precisely — the root set, the mark closure, the sweep, and the soundness argument that makes GC observably a no-op — then maps every piece to the exact file, function, and (for the interactive mark-phase animation) UI state machine that implements it.

Contents

1 Why a garbage collector is needed at all **1**

2 The algorithm: reachability closure, then sweep **2**
2.1 The store and its roots . . . 2
2.2 Mark: the reachability closure . . . 3
2.3 Sweep . . . 3
2.4 The composed transition . . . 3

3 Correctness **4**
3.1 Soundness: GC never collects a live object . . . 4
3.2 Semantic transparency . . . 4
3.3 No write barrier, no tri-color invariant . . . 4

4 Implementation mapping **4**
4.1 `reachability.ts`: the steppable mark phase . . . 5
4.2 `gc.ts`: sweep and the batch entry point . . . 5
4.3 `stepperPanel.ts`: the interactive driver . . . 6
4.4 `test/gc.entry.ts`: the property-testing harness . . . 6

5 Verification **7**

6 Theoretical sources **7**

Note on scope **7**

1 Why a garbage collector is needed at all

Block-MiniJava's two value models make fundamentally different storage commitments (design note §2–§4; the sibling note `typing-and-evaluation-rules.tex`, §4.2):

- **Model A** (faithful MiniJava) represents every object and array as a *heap-allocated* cell reached only through a Ref(ℓ) value. This is what makes Java-style aliasing and `field = e` mutation observable in the first place, and it is exactly what makes the heap a genuine, unboundedly growing resource: `new` never runs out on its own, and nothing frees a cell when it becomes unreachable.
- **Model B** (structure-preserving) has *no heap at all* — objects and arrays are inline values, copied by binding. There is nothing to collect, and this GC module is not exercised, wired, or meaningful under Model B.

So garbage collection in this project is specifically *Model A's* concern: a way to reclaim heap cells the running program can no longer reach, without changing what the program computes or prints.

2 The algorithm: reachability closure, then sweep

2.1 The store and its roots

Let $H : \mathrm{Loc} \rightharpoonup \mathrm{HeapObj}$ be the heap (`MachineState.heap`, `minijavaMachine.ts:130`) and let a *live configuration* be the full machine state $\Sigma = \langle C, \Phi, H \rangle$ (control, frame stack, store; §4.2 of the companion evaluation-rules note). A location $\ell \in \mathrm{dom}(H)$ is **garbage** exactly when it is not reachable from anything a future step of the program could still read. Three, and only three, places pin a heap location outside H itself:

1. **Environment roots** — every value directly bound in a *live* frame's locals ρ, plus that frame's this (a Ref under Model A). **Every** frame on the call stack is a root source, not just the top one: a MiniJava call stack can hold several active frames (nested method calls), and a parent frame paused mid-call still owns live bindings that must not be collected out from under it.
2. **Kontinuation roots** — every value already computed and stashed inside a *pending continuation* of a live frame, e.g. the left operand held by KBinR, the array already looked up by KLookupIdx, or the receiver and already-evaluated arguments held by KCallArgs. These are live exactly as much as a local variable is — they are simply mid-computation rather than named.
3. **The in-flight control value** — if $C = \mathrm{Value}(v)$, v itself is a root. This is the subtlest of the three: right after a NEW transition, the fresh Ref exists *only* as the machine's in-flight control value for exactly one step, before the pending KAssign or KCallArgs continuation consumes it into a binding. A collector that only scanned frames (case 1–2) and happened to run in that one-step window would reclaim an object the very next step needs — so this case is not optional, and both the library and the UI driver include it explicitly (§4).

Formally, the root set is

$$\mathrm{Roots}(\Sigma) \;=\; \bigcup_{\varphi \in \Phi} \big(\mathrm{addrs}(\rho_\varphi) \cup \{\mathrm{Ref}(\mathsf{self}_\varphi)\} \big) \;\cup\; \bigcup_{\varphi \in \Phi} \mathrm{addrs}(\kappa_\varphi) \;\cup\; \begin{cases} \mathrm{addrs}(v) & C = \mathrm{Value}(v) \\ \varnothing & \text{otherwise} \end{cases}$$

where addrs(v) collects every Ref a value directly or transitively contains (a Ref contributes itself; an inline Obj/Arr — which does not occur under Model A's own values, but is handled defensively — contributes its fields/elements recursively).

2.2 Mark: the reachability closure

Reachability is the least fixed point of the root set closed under following object fields and array elements:

$$\frac{\ell \in \mathrm{Roots}(\Sigma)}{\ell \in \mathrm{reach}(\Sigma)} \ (\textsc{Mark-Root})$$

$$\frac{\ell \in \mathrm{reach}(\Sigma) \quad H(\ell) = \mathrm{Obj}(C, \overline{f{=}v}) \quad \ell' \in \mathrm{addrs}(v_i) \text{ for some } f_i}{\ell' \in \mathrm{reach}(\Sigma)} \ (\textsc{Mark-Field})$$

$$\frac{\ell \in \mathrm{reach}(\Sigma) \quad H(\ell) = \mathrm{Arr}(\overline{v}) \quad \ell' \in \mathrm{addrs}(v_i) \text{ for some } v_i}{\ell' \in \mathrm{reach}(\Sigma)} \ (\textsc{Mark-Elem})$$

This is a standard *tracing* collector's mark phase: a graph traversal of the heap starting from the roots, following every outgoing edge (field or element) until no new node is discovered. **Dangling references** ($\ell \in \mathrm{reach}(\Sigma)$ but $\ell \notin \mathrm{dom}(H)$) terminate that branch of the traversal rather than erroring — they cannot occur reachably in this machine (there is no explicit deallocation before GC runs, so every live Ref points somewhere), but the traversal is defensive about it regardless. **Cycles** terminate because each location is expanded into its own outgoing edges *at most once*, however many times it is rediscovered — the standard visited-set guard that makes tracing GC total on a possibly-cyclic object graph (an object graph with a field cycle, e.g. two objects each holding a reference to the other, is completely ordinary in Java and must not loop the collector forever).

2.3 Sweep

Sweep is a pure filter: keep exactly the store entries whose address survived marking, and drop the rest.

$$\mathrm{sweep}(H, R) \ = \ \{\, \ell \mapsto H(\ell) \mid \ell \in \mathrm{dom}(H) \cap R \,\}$$

Two properties are worth stating explicitly because they are what make sweep *safe* to run at arbitrary points in execution rather than only between well-chosen safepoints:

- **Non-moving.** A surviving object keeps its address ℓ and is the *same* object (not a copy) — sweep never relocates anything, so every $\mathrm{Ref}(\ell)$ still held anywhere in the machine (by construction, only in roots or reachable objects, both of which survive) remains valid without any forwarding-pointer or pointer-rewriting pass. This is what a *moving* (e.g. copying, compacting) collector would need and this one deliberately avoids, at the cost of not compacting fragmented free space — an acceptable trade for a pedagogical/visualization machine, not a performance-oriented one.
- **Monotone addresses.** Sweep only ever *shrinks* the domain of H; it never reassigns or recycles a retired address. The machine's separate allocation counter nextLoc is untouched by GC and keeps incrementing monotonically, so a freshly allocated object can never collide with the address of something GC just swept away, even one step later.

2.4 The composed transition

$$\frac{R = \mathrm{reach}(\Sigma) \quad H' = \mathrm{sweep}(H, R)}{\langle C, \Phi, H \rangle \ \longrightarrow_{\mathrm{GC}} \ \langle C, \Phi, H' \rangle} \ (\mathrm{GC})$$

$\longrightarrow_{\mathrm{GC}}$ is a transition on the *same* kind of configuration the evaluator's $\longrightarrow$ relation (the companion note's §4.2) operates on, but it is a genuinely separate relation: it never changes C or Φ, is not one of the named evaluation rules (NEW, CALL, ...), and — critically — *runs to completion in*

one shot. There is no partially-marked, partially-swept state that a caller (or, under Model A, the running mutator) can ever observe: §3.3 explains why that lets this collector skip machinery that a concurrent or incremental collector would need.

3 Correctness

3.1 Soundness: GC never collects a live object

By construction, reach(Σ) is downward closed under the exact three places a running program can ever *read* a Ref from next: a frame's locals/`this`, a frame's pending continuations, and the in-flight control value. Any address the program could dereference on its very next step is therefore in Roots(Σ) $\subseteq$ reach(Σ), and any address reachable from a root by following live fields/elements is in reach(Σ) by the closure rules — so sweep(H, reach(Σ)) can never drop something the next step needs. The one case that would silently break this argument (the in-flight Value(v) root, §2 case 3) is exactly the case both the library's caller contract and the UI driver call out and implement — see §4 and the root-collection code at `stepperPanel.ts:649` / `test/gc.entry.ts:79`, whose comments both point at the identical hazard.

3.2 Semantic transparency

The stronger, tested property is that GC is *observably a no-op*: for any program p that terminates,

$$\text{output}(\text{run}(p)) \;=\; \text{output}(\text{run}_{\text{GC}}(p))$$

where run steps to completion with GC never invoked and run_{GC} composes a full mark-and-sweep pass after *every single step* (the most aggressive schedule possible, and hence the strongest test of the property) — even though the two runs' final heaps, and every intermediate heap, generally differ. This is checked, not just argued, by `test/run_gc.js`'s integration layer (§5): running GC after every step never changes final status, output, or step count on programs deliberately built to generate garbage mid-run (aliasing plus loop-generated garbage, a field-chain rebind, and an array resize that abandons the old backing array).

3.3 No write barrier, no tri-color invariant

A collector that can be *paused* mid-mark while the mutator keeps running needs to maintain an invariant relating collector and mutator state — classically Dijkstra et al.'s tri-color invariant, enforced by a *write barrier* that intercepts every mutator pointer write so a black (already-scanned) object can never come to point at a white (not-yet-scanned) one without the collector noticing [3]. Block-MiniJava's collector needs none of this, because it is never actually concurrent with the mutator: `runGC` (`gc.ts:52`) computes the *entire* reachable set before sweeping, in one synchronous call, and every caller — the pure test loop in `test/gc.entry.ts` and the interactive UI driver in `stepperPanel.ts` — enforces *never interleaved with a mutator step* as an invariant (§4). The mark phase's *visual animation* (one heap box highlighted per timer tick, §4.3) only *looks* incremental to the user; the underlying machine state is frozen for its whole duration, so there is no window in which a mutator step could invalidate an in-progress mark. This is a deliberate simplicity trade documented directly in the source (`gc.ts:10--17`): stop-the-world tracing GC is exactly the right amount of collector for a machine whose entire point is single-stepped, observed execution, not concurrent throughput.

4 Implementation mapping

The collector is deliberately split into three layers with three different responsibilities, none of which imports `step` or is imported by it:

Layer	File	Responsibility
Reachability (pure, testable in isolation)	`src/core/semantics/reachability.ts`	The mark closure over hand-built `MachineValue`/heap fixtures — no dependency on `MachineState`, `Frame`, or Blockly.
Collector (pure, composes reachability + sweep)	`src/core/semantics/gc.ts`	`sweep` and `runGC`: the batch, non-animated mark-then-sweep entry point a test or a future non-visual caller uses.
Interactive driver (impure, owns the animation and the UI's GC state)	`src/core/ui/stepperPanel.ts`	Extracts roots from a real `MachineState`, drives `markPhase` one event per animation tick, renders the mark/sweep, and commits the swept heap as a new history entry.

4.1 `reachability.ts`: the steppable mark phase

Symbol	Role
`Address = Loc` (l.32)	Type alias; a heap address is the same `Loc` the machine already uses.
`MarkSource` (l.35–37)	Tags *why* an address was just marked — a root (`'env'` or `'kont'` origin) or reached via another address — purely for the UI's provenance/animation text, not for correctness.
`MarkEvent` (l.40–43)	One "object newly marked" record: the address and its `MarkSource`.
`markPhase(env, store, kont)` (l.61–96)	The traversal core: a JavaScript *generator*, so a caller can resume it one `.next()` at a time. Internally an explicit worklist (not the call stack, so it cannot stack-overflow on a deep or cyclic heap), a `marked` set that both prevents re-expansion and gives the Mark-Root/Mark-Field/Mark-Elem rules their cycle-safety, and `collectAddresses` (l.114–130) walking one value down to the `Ref`s it contains.
`reachableAddresses(env, store, kont)` (l.103–111)	The batch form: drains `markPhase` into a `Set`. **There is exactly one traversal implementation** — this is not a second algorithm, just a caller that does not care about visitation order. Used by tests and by `gc.ts`.

4.2 `gc.ts`: sweep and the batch entry point

Symbol	Role
`sweep(store, reachable)` (l.38–44)	The sweep phase exactly as §2: a fresh `Map` containing only the reachable entries; surviving values are the *same* `HeapObj` references (no copying), matching the non-moving property.
`runGC(env, store, kont)` (l.52–59)	Mark to completion via `reachableAddresses`, then sweep. This is $\longrightarrow_{\text{GC}}$ itself, as a pure function on `(roots, heap)` rather than on a full `MachineState` — deliberately, so it stays independent of which caller (test loop or UI) is responsible for extracting roots from a particular state shape.

4.3 `stepperPanel.ts`: the interactive driver

This is the "real, running" GC integration — the `Run GC` button and the auto-trigger on the CESK tab (`index.html:294--300`).

Function / state	Role
`kontRoots(kont)` (l.608–632)	Enumerates exactly the `Kont` tags that pin a `MachineValue` (`KArrAssignVal.index`, `KBinR.left`, `KLookupIdx.arr`, `KCharAtIdx.str`, `KCallArgs.recv/.done`) — the concrete realization of the "kontinuation roots" case in §2. Every other `Kont` tag carries only `Blockly.Block`/string/null control-flow bookkeeping, nothing rooted.
`frameRoots(frame)` (l.634–638)	One frame's env roots (its `locals` plus, under Model A, `REF_V(frame.self)` for `this`) and kont roots.
`rootsOf(state)` (l.649–659)	Flattens *every* frame in `state.stack` (not just the top) via `frameRoots`, and adds `state.control.value` when `control.tag === 'Value'` — the exact three root sources of §2, spelled out against the real `MachineState` shape. The in-flight-value comment at l.640–648 is the same hazard §3 names.
`startGCAnimation()` (l.703–725)	Entry point for both the manual button and the auto-trigger. Computes `rootsOf(current)`, opens a `markPhase` generator, and drains it *one event per `setInterval` tick* (`GC_MARK_INTERVAL_MS = 300`ms, l.35) purely for the visualization — `stopPlay()` is called first so the mutator's own Play loop cannot interleave.
`flashMarkedBox(loc)` (l.664–670)	Per-tick visual feedback: the just-marked heap box flashes (`is-gc-marked`).
`finishSweep(marked)` (l.676–697)	Once the generator is exhausted: grey out every heap box *not* in `marked`, wait `GC_SWEEP_FADE_MS = 450`ms for the fade, then commit — pushes the pre-GC state onto the same `history` stack the mutator's `Back` button uses (so GC is undoable exactly like a step), computes the swept heap via `sweep` (the same function `gc.ts` exports), and clears `lastEffect`/`lastRule` so a surviving box does not spuriously re-flash as "just written."
`autoGcEnabled`, `gcThreshold` (l.53–54)	Persisted (`localStorage`) settings for the automatic trigger, read/written by `loadGcSettings`/`updateAutoGcEnabled`/`updateGcThreshold` (l.746–765).
`stepOnce()`'s auto-trigger (l.812–819)	After a mutator step, if auto-GC is on *and* the step's effect was a `'new'` *and* the heap has grown past `gcThreshold`, immediately calls `startGCAnimation()` — so collection is allocation-triggered, the classic GC scheduling policy, rather than time-triggered.
`gcAnimation` guard (l.56, checked at l.211,215,704,800,823)	Non-null for the whole duration of a mark-then-sweep pass; every mutator control (`Step`, `Back`, `Play`) is disabled while it is set, enforcing "GC and mutator steps are never interleaved" at the UI layer, on top of `gc.ts`'s own single-call atomicity.

4.4 `test/gc.entry.ts`: the property-testing harness

A fourth, test-only composition (`collectGarbage`, l.95–99; `runWithGCEveryStep`/`runWithoutGC`, l.131–155) reproduces `stepperPanel.ts`'s `rootsOf` logic against a real `MachineState` (not hand-built fixtures) and threads `runGC` directly into the step loop, once per step, to produce the two traces §3's semantic-transparency property compares.

5 Verification

- `test/run_reachability.js` (14 cases) — pure fixtures for `reachableAddresses`/`markPhase`: objects reachable only through a field of another object, only through an array element, only through a returned frame (unreachable), only through a pending continuation (reachable); unreachable and reachable A↔B reference cycles; that `reachableAddresses` never mutates the store; empty root sets reaching nothing; and step-by-step assertions on `markPhase`'s generator protocol itself (first/second/third `.next()`, a kont-pinned root's reported origin, and that draining the generator matches the batch function).

- `test/run_gc.js` (14 cases) — (a)/(b) pure `sweep`/`runGC` unit tests on hand-built stores (reachable objects survive *unchanged*, unreachable ones are gone); (c) the semantic-transparency property (§3) on real programs: aliasing plus loop-generated garbage, a field-chain rebind, and an array-resize scenario, each checked three ways — GC-on vs. GC-off give an identical observable result, the specific object/value that should survive does, and the heap actually shrinks under GC (so the test cannot pass vacuously because GC happened to collect nothing).

- **CESK panel smoke test** (`test/smoke-csesk.entry.ts`, part of `npm test`'s browser-machine-panel suite) exercises a real mid-run GC through the UI driver end to end: allocation count before/after, the specific soon-to-be-garbage cell present then absent, the heap shrinking by exactly the swept count, the status text reporting a GC summary (not a step count), and the program still finishing — with the right output — after a mid-run collection.

Together these three suites test the collector at three levels: the raw graph algorithm on synthetic heaps, the composed mark+sweep+transparency property on real programs run headlessly, and the full interactive mark-then-sweep pass driven exactly as a user would trigger it.

6 Theoretical sources

- **Mark-and-sweep** is McCarthy's original algorithm, introduced for automatic storage reclamation in Lisp [1]: mark every cell reachable from a root set, then reclaim every unmarked cell. This is, unmodified, the two-phase structure of `runGC`.

- **Root sets, tracing vs. reference counting, and the stop-the-world/incremental/concurrent design space** are given their standard modern treatment in Jones, Hosking, and Moss's *The Garbage Collection Handbook* [2] — the vocabulary ("roots," "mutator," "non-moving," "stop-the-world") used throughout this note follows that reference.

- **The tri-color invariant and write barriers**, which this collector deliberately does *not* need (§3.3), are Dijkstra, Lamport, Martin, Scholten, and Steffens's on-the-fly garbage collection technique for a collector running *concurrently* with the mutator [3] — included here as the contrasting case that explains, by what it avoids, why this simpler stop-the-world collector is sufficient.

Note on scope

This collector targets Model A only; it is meaningless (and not wired up) under Model B, which has no heap. It is also non-generational and non-compacting by design — the project's goal is a legible, animatable collection pass over a small, interactively-stepped heap, not throughput on a large one.

References

[1] J. McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4):184–195, 1960.

[2] R. Jones, A. Hosking, and E. Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. CRC Press, 2011.

[3] E. W. Dijkstra, L. Lamport, A. J. Martin, C. S. Scholten, and E. F. M. Steffens. On-the-fly garbage collection: an exercise in cooperation. *Communications of the ACM*, 21(11):966–975, 1978.